ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		К.А. Кочин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

Освоение работы с программируемыми шейдерами

по курсу: КОМПЬЮТЕРНАЯ ГРАФИКА

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Задание

Выполнить на выбор один из вариантов:

- Шейдер с сохранением данных в текстуре,
- Процедурный, текстурный шейдер,
- Традиционные шейдер,
- Фрагментный шейдер,
- Вершинный шейдер.

Примеры из пакета с компилятором Cg не допускаются

2. Текст программы

```
#define WIN32_LEAN_AND_MEAN 1
#include <windows.h>

#include "common.h"
#include "math/math3d.h"
#include "math/mathgl.h"
#include "OpenGL.h"
#include "GLWindow.h"
#include "Shader.h"
#include "Texture.h"
#include "Mesh.h"
#include "Camera.h"
#include "Light.h"
#include "Material.h"

// вспомогательный макрос
#define LOAD_SHADER(name) \
      ShaderProgramCreateFromFile("data/" name ".vs", "data/" name ".fs")

// индекс шейдерной программы
static GLuint depthProgram = 0, quadProgram = 0, shadowmapProgram = 0;

// индексы текстур
static GLuint colorTexture = 0, depthTexture = 0;

// индекс FBO
static GLuint depthFBO = 0;

// положение курсора и его смещение с последнего кадра
static int cursorPos[2] = {0,0}, rotateDelta[2] = {0,0}, moveDelta[2] = {0,0};

static const uint32_t meshCount = 3;

static Mesh     meshes[meshCount], quadMesh;
static Material materials[meshCount], quadMaterial;

static float3 torusRotation = {0.0f, 0.0f, 0.0f};

static Light  directionalLight;
static Camera mainCamera, quadCamera, lightCamera;

static bool doRenderQuad = false;

// инициализаця OpenGL
bool GLWindowInit(const GLWindow &window)
```

```
{
	// спрячем курсор
	InputShowCursor(false);

	// устанавливаем вьюпорт на все окно
	glViewport(0, 0, window.width, window.height);

	// параметры OpenGL
	glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
	glClearDepth(1.0f);
	glEnable(GL_DEPTH_TEST);
	glEnable(GL_CULL_FACE);

	// создадим и загрузим шейдерные программы
	if ((depthProgram = LOAD_SHADER("depth")) == 0
		|| (quadProgram = LOAD_SHADER("quad")) == 0
		|| (shadowmapProgram = LOAD_SHADER("shadowmap")) == 0)
	{
		return false;
	}

	// настроим направленный источник освещения
	LightDefault(directionalLight, LT_DIRECTIONAL);
	directionalLight.position.set(3.0f, 3.0f, 3.0f, 0.0f);

	// загрузим текстуры
	colorTexture = TextureCreateFromTGA("data/texture.tga");

	// создадим текстуру для хранения глубины
	depthTexture = TextureCreateDepth(window.width * 2, window.height * 2);

	// создадим примитивы и настроим материалы
	// плоскость под вращающимся тором
	MeshCreatePlane(meshes[0], vec3(0.0f, -1.6f, 0.0f), 30.0f);
	MaterialDefault(materials[0]);
	materials[0].texture = colorTexture;
	materials[0].diffuse.set(0.3f, 1.0f, 0.5f, 1.0f);

	// кубик
	MeshCreateCube(meshes[1], vec3(0.0f, 1.2f, 0.0f), 1.0f);
	MaterialDefault(materials[1]);
	materials[1].texture = colorTexture;
	materials[1].diffuse.set(0.3f, 0.5f, 1.0f, 1.0f);
	materials[1].specular.set(0.8f, 0.8f, 0.8f, 1.0f);
	materials[1].shininess = 20.0f;

	// тор, присоединенный к кубику сверху
	MeshCreateTorus(meshes[2], vec3(4.0f, 2.2f, 0.0f), 0.6f);
	MaterialDefault(materials[2]);
	materials[2].texture = colorTexture;
	materials[2].diffuse.set(1.0f, 0.5f, 0.3f, 1.0f);
	materials[2].specular.set(0.8f, 0.8f, 0.8f, 1.0f);
	materials[2].shininess = 20.0f;

	// настроим полноэкранный прямоугольник
	MeshCreateQuad(quadMesh, vec3(0.0f, 0.0f, 0.0f), 1.0f);
	quadMesh.rotation = mat3(GLRotationX(90.0f));
	MaterialDefault(quadMaterial);
	quadMaterial.texture = depthTexture;

	// создадим и настроим камеру
	const float aspectRatio = (float)window.width / (float)window.height;
	CameraLookAt(mainCamera, vec3(-5.0f, 10.0f, 10.0f), vec3_zero, vec3_y);
	CameraPerspective(mainCamera, 45.0f, aspectRatio, 0.5f, 100.0f);
```

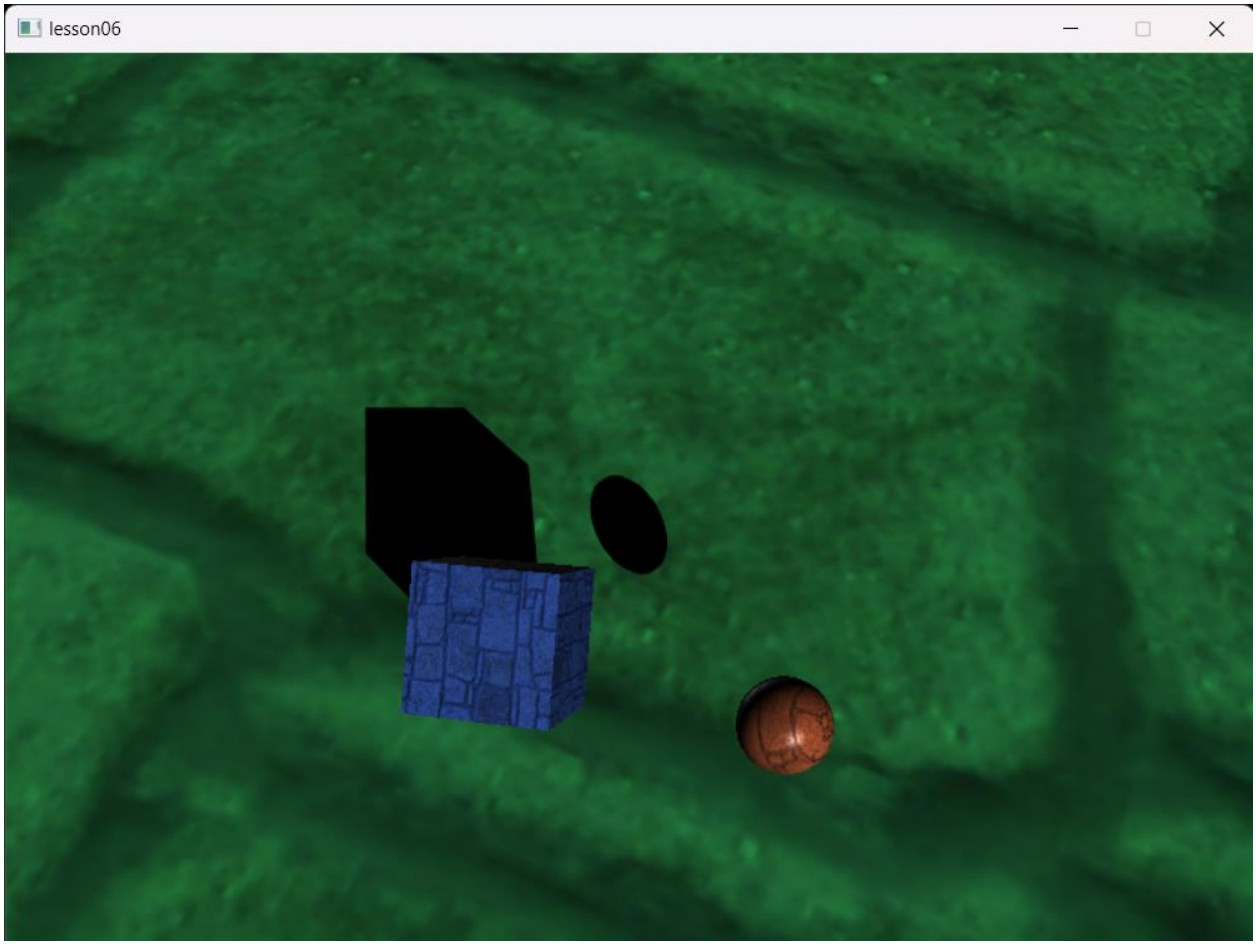
```
	// камера источника света
	CameraLookAt(lightCamera, directionalLight.position, -
directionalLight.position, vec3_y);
	CameraOrtho(lightCamera, -5.0f, 5.0f, -5.0f, 5.0f, -10.0f, 10.0f);

	// камера полноэкранного прямоугольника, для рендера текстуры глубины
	CameraLookAt(quadCamera, vec3_zero, -vec3_z, vec3_y);
	CameraOrtho(quadCamera, -1.0f, 1.0f, -1.0f, 1.0f, -1.0f, 1.0f);

	// создаем FBO для рендера глубины в текстуру
	glGenFramebuffers(1, &depthFBO);
	// делаем созданный FBO текущим
	glBindFramebuffer(GL_FRAMEBUFFER, depthFBO);

	// отключаем вывод цвета в текущий FBO
	glDrawBuffer(GL_NONE);
	glReadBuffer(GL_NONE);

	// указываем для текущего FBO текстуру, куда следует производить рендер
глубины
	glFramebufferTexture(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, depthTexture, 0);

	// проверим текущий FBO на корректность
	GLenum fboStatus;
	if ((fboStatus = glCheckFramebufferStatus(GL_FRAMEBUFFER)) !=
GL_FRAMEBUFFER_COMPLETE)
	{
		LOG_ERROR("glCheckFramebufferStatus error 0x%X\n", fboStatus);
		return false;
	}

	glBindFramebuffer(GL_FRAMEBUFFER, 0);

	// проверим не было ли ошибок
	OPENGL_CHECK_FOR_ERRORS();

	return true;
}

// очистка OpenGL
void GLWindowClear(const GLWindow &window)
{
	(void)window;

	for (uint32_t i = 0; i < meshCount; ++i)
		MeshDestroy(meshes[i]);

	ShaderProgramDestroy(depthProgram);
	ShaderProgramDestroy(quadProgram);
	ShaderProgramDestroy(shadowmapProgram);

	glBindFramebuffer(GL_FRAMEBUFFER, 0);
	glDeleteFramebuffers(1, &depthFBO);

	TextureDestroy(colorTexture);
	TextureDestroy(depthTexture);

	InputShowCursor(true);
}

void RenderScene(GLuint program, const Camera &camera)
{
	// делаем шейдерную программу активной
	ShaderProgramBind(program);
```

```
	LightSetup(program, directionalLight);
	CameraSetupLightMatrix(program, lightCamera);

	TextureSetup(program, 1, "depthTexture", depthTexture);
	glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_COMPARE_MODE,
GL_COMPARE_REF_TO_TEXTURE);

	for (uint32_t i = 0; i < meshCount; ++i)
	{
		CameraSetup(program, camera, MeshGetModelMatrix(meshes[i]));
		MaterialSetup(program, materials[i]);
		MeshRender(meshes[i]);
	}
}

void RenderQuad(GLuint program, const Camera &camera)
{
	// делаем шейдерную программу активной
	ShaderProgramBind(program);

	CameraSetup(program, camera, MeshGetModelMatrix(quadMesh));
	MaterialSetup(program, quadMaterial);
	glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_COMPARE_MODE, GL_NONE);
	MeshRender(quadMesh);
}

// функция рендера
void GLWindowRender(const GLWindow &window)
{
	glBindFramebuffer(GL_FRAMEBUFFER, depthFBO);
	glViewport(0, 0, window.width * 2, window.height * 2);
	glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE);
	glDepthMask(GL_TRUE);
	glClear(GL_DEPTH_BUFFER_BIT);
	glCullFace(GL_FRONT);

	RenderScene(depthProgram, lightCamera);

	glBindFramebuffer(GL_FRAMEBUFFER, 0);
	glViewport(0, 0, window.width, window.height);
	glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE);
	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
	glCullFace(GL_BACK);

	if (doRenderQuad)
		RenderQuad(quadProgram, quadCamera);
	else
		RenderScene(shadowmapProgram, mainCamera);

	// проверка на ошибки
	OPENGL_CHECK_FOR_ERRORS();
}

// функция обновления
void GLWindowUpdate(const GLWindow &window, double deltaTime)
{
	ASSERT(deltaTime >= 0.0); // проверка на возможность бага

	(void)window;

	// Углы поворота для кубика
static float3 cubeRotation = {0.0f, 0.0f, 0.0f};

// Углы поворота для тора
static float3 torusRotation = {0.0f, 0.0f, 0.0f};
```

```
if ((cubeRotation[0] += 30.0f * (float)deltaTime) > 360.0f)
    cubeRotation[0] -= 360.0f;

if ((cubeRotation[1] += 15.0f * (float)deltaTime) > 360.0f)
    cubeRotation[1] -= 360.0f;

if ((cubeRotation[2] += 7.0f * (float)deltaTime) > 360.0f)
    cubeRotation[2] -= 360.0f;

// Задаем матрицу вращения кубика
meshes[1].rotation = GLFromEuler(cubeRotation[0], cubeRotation[1], cubeRotation[2]);

// Углы поворота тора с учетом углов поворота кубика
torusRotation[0] = cubeRotation[0];
torusRotation[1] = cubeRotation[1];
torusRotation[2] = cubeRotation[2];

// Задаем матрицу вращения тора
meshes[2].rotation = GLFromEuler(torusRotation[0], torusRotation[1],
torusRotation[2]);


      // вращаем камеру
      CameraRotate(mainCamera, (float)deltaTime * rotateDelta[1], (float)deltaTime
* rotateDelta[0], 0.0f);
      // двигаем камеру
      CameraMove(mainCamera, (float)deltaTime * moveDelta[0], 0.0f,
(float)deltaTime * moveDelta[1]);

      rotateDelta[0] = rotateDelta[1] = 0;
      moveDelta[0] = moveDelta[1] = 0;

      OPENGL_CHECK_FOR_ERRORS();
}

// функция обработки ввода с клавиатуры и мыши
void GLWindowInput(const GLWindow &window)
{
      // центр окна
      int32_t xCenter = window.width / 2, yCenter = window.height / 2;

      // выход из приложения по кнопке Esc
      if (InputIsKeyPressed(VK_ESCAPE))
            GLWindowDestroy();

      if (InputIsKeyPressed(VK_SPACE))
            doRenderQuad = !doRenderQuad;

      // переключение между оконным и полноэкранным режимом
      // осуществляется по нажатию комбинации Alt+Enter
      if (InputIsKeyDown(VK_MENU) && InputIsKeyPressed(VK_RETURN))
      {
            GLWindowSetSize(window.width, window.height, !window.fullScreen);
            //InputShowCursor(window.fullScreen ? false : true);
      }

      moveDelta[0] = 10 * ((int)InputIsKeyDown('D') - (int)InputIsKeyDown('A'));
      moveDelta[1] = 10 * ((int)InputIsKeyDown('S') - (int)InputIsKeyDown('W'));

      InputGetCursorPos(cursorPos, cursorPos + 1);
      rotateDelta[0] += cursorPos[0] - xCenter;
      rotateDelta[1] += cursorPos[1] - yCenter;
      InputSetCursorPos(xCenter, yCenter);
}
```

```
int WINAPI WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
{
	int result;

	LoggerCreate("lesson06.log");

	if (!GLWindowCreate("lesson06", 800, 600, false))
		return 1;

	result = GLWindowMainLoop();

	GLWindowDestroy();
	LoggerDestroy();

	return result;
}
```

3. Скриншоты выполнения работы



4. Вывод

В представленном коде использованы следующие типы шейдеров:

1) Вершинный шейдер (Vertex Shader): Вершинный шейдер обрабатывает вершины трехмерных объектов.

2) Фрагментный шейдер (Fragment Shader): Фрагментный шейдер отвечает за окраску пикселей на экране. В коде присутствует фрагментный шейдер, используемый в RenderQuad и RenderScene, чтобы применить текстуры и эффекты материалов на трехмерные объекты.

3) Шейдер для создания карты глубины (Depth Shader): Этот шейдер, представленный в depthProgram, используется для создания карты глубины (shadow map) при первом проходе рендеринга сцены. Он записывает информацию о глубине сцены в текстуру depthTexture, чтобы создать эффект теней.

4) Текстурный шейдер (Texture Shader): Шейдеры, используемые в RenderScene и RenderQuad, могут считаться текстурными шейдерами, так как они применяют текстуры к поверхностям объектов.

Таким образом, код включает в себя различные типы шейдеров, используемых для реализации различных аспектов рендеринга трехмерной сцены, включая создание теней и наложение текстур.